

## EE345 Mod2-Lec4: Minimum-Norm Solutions

# Overview

## 1. Module 1: Distances and the Algorithms they Enable

- ▶ Distances and metrics: Euclidean distance, Manhattan distance, cosine distance, worst-case distance, etc.
- ▶ Vectors and matrices: dot product, matrix multiplication, transpose, inverse, etc.
- ▶ Distance based algorithms: k-nearest neighbors (supervised), k-means (unsupervised)

## 2. Module 2: Linear Models & Regression

- ▶ Convex costs: quadratic functions/forms
- ▶ Linear models and their prevalence in machine learning
- ▶ Least squares: quadratic cost, linear regression
- ▶ **Minimum-norm solutions and introduction to regularization**

## 3. Module 3: Gradient Descent, Regularization, Kernel Methods

# Minimum-Norm Problem

- We saw how to obtain the least squares approximate solution to  $X\theta = y$  corresponding to an over-determined set of equations—i.e.,

data matrix  $X \in \mathbb{R}^{n \times d}$  with  $n \gg d$  ("tall" matrix).

- Here, there is rarely an exact solution to  $X\theta = y$ , and instead we find an **approximate solution** by minimizing  $\|X\theta - y\|^2$ .

# Underdetermined Systems

An *underdetermined least squares problem* arises when there are fewer equations than unknowns ( $n < d$  i.e.,  $X$  is a “wide” matrix):

$$X = \begin{bmatrix} | & | & \cdots & | \\ x_1 & x_2 & \cdots & x_d \\ | & | & \cdots & | \end{bmatrix}$$

in which case there are infinitely many solutions to  $X\theta = y$ .

Intuition: too many features can cause multiple solutions...

# Minimum-Norm Problem

$$\min_{\theta} \|\theta\|_2^2 \text{ subject to } X\theta = y. \quad (\text{min-norm-problem})$$

When does this problem arise and why do we care?

# From Classical to Modern ML

- **Classical view:** Too few parameters  $\Rightarrow$  *underfitting*.
- **Modern reality:** *Overparameterization* occurs in models with **more parameters than data**.
- Deep nets with millions of weights can *interpolate* (zero training error) yet still *generalize*.
- Key question: *Of the many solutions, which one is "stable" (i.e. robust to small changes in the data)? And, What selection method produces a "good" fit?*

# Real-World Examples of Underdetermined Systems

## Deep Neural Networks.

- ResNet-50 has about 25 million parameters, while a medical imaging dataset may have only a few thousand training examples.
- Despite  $d \gg n$ , gradient-based training often achieves zero training error.
- The solution behaves as if it were a *minimum-norm* solution in function space, illustrating the **implicit regularization** of **stochastic gradient descent (SGD)**.

## Learning word embeddings or language models.

- In word2vec or transformer pretraining, the number of parameters in the embedding or attention layers greatly exceeds the number of distinct word co-occurrences observed per batch.
- Weight decay or early stopping biases the solution toward low-norm parameter matrices.

# Real-World Examples of Underdetermined Systems

**Genomics and Biomedical Prediction.** Consider predicting disease risk from gene expression.

- A typical dataset may have  $n = 100$  patients but  $d = 20\text{k}$  gene features.
- **Ridge regression** makes the problem well-posed:

$$\min_{\theta} \|X\theta - y\|_2^2 + \lambda \|\theta\|_2^2,$$

yielding a minimum-norm solution that stabilizes predictions.

- This also occurs in *bag-of-words* text classification tasks.



# Real-World Examples of Underdetermined Systems

## Redundant robot arm (inverse kinematics).

- A 7-DOF robotic arm can position its end-effector in 3D, creating redundancy:

$$J(\theta)\dot{\theta} = \dot{x}, \quad J \in \mathbb{R}^{3 \times 7}.$$

- The minimum-norm velocity

$$\dot{\theta} = J^{\top}(JJ^{\top})^{-1}\dot{x}$$

achieves the desired motion with minimal total joint movement, leading to smoother, energy-efficient trajectories.

## Learning a robot control policy from demonstrations.

- A neural policy network may contain millions of parameters but only thousands of example trajectories.
- Weight decay or early stopping biases learning toward a minimum-energy control policy that matches the expert behavior without overfitting.

# Problem: Finding the Minimum Norm Solution

We want to find the smallest-norm vector  $\theta$  that satisfies the linear system

$$X\theta = y,$$

where  $X \in \mathbb{R}^{n \times d}$  has full row rank ( $n < d$ ) and the system is consistent. This problem is underdetermined and can be cast as the following optimization problem:

$$\min_{\theta} \|\theta\|^2 \quad \text{s.t. } X\theta = y.$$

# Problem: Finding the Minimum Norm Solution

**Part (a):** Instead of solving the constrained problem, consider the augmented objective

$$J(\theta) = \frac{1}{2}\|\theta\|_2^2 + \lambda^\top(y - X\theta), \quad \text{where } \lambda \in \mathbb{R}^m \text{ is a fixed vector.}$$

Compute  $\nabla_\theta J(\theta)$  and find  $\theta$  that minimizes  $J(\theta)$  in terms of  $\lambda$ .

# Problem: Finding the Minimum Norm Solution

# Problem: Finding the Minimum Norm Solution

**Part (b):** Substitute the result into  $X\theta = y$  and solve for  $\lambda$  in terms of  $X$  and  $y$ .

# Problem: Finding the Minimum Norm Solution

**Part (c):** Substitute  $\lambda$  back to get an explicit expression for  $\theta$ . Show that

$$\hat{\theta} = X^T (X X^T)^{-1} y.$$

# Problem: Finding the Minimum Norm Solution

**Part (d):** Explain why this  $\hat{\theta}$  is the same as the solution to

$$\min_{\theta} \|\theta\|_2 \quad \text{s.t. } X\theta = y.$$

# Problem: Finding the Minimum Norm Solution



# Problem: Finding the Minimum Norm Solution

# Problem: Finding the Minimum Norm Solution

**Part (e):** Let

$$X = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \quad \text{and} \quad y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Find the minimum norm solution to  $X\theta = y$ .

# Problem: Finding the Minimum Norm Solution

So, why do we choose the minimum norm solution then? Coming back to the key questions at the top...

# Setup: Underdetermined Least Squares

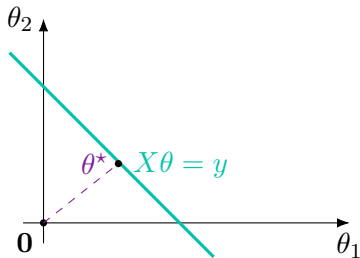
- Suppose  $X \in \mathbb{R}^{n \times d}$  with  $n < d$  and  $\text{rank}(X) = n$ .

# The Minimum-Norm Interpolant

- Define

$$\theta^* = \arg \min_{\theta} \|\theta\|_2^2 \quad \text{s.t.} \quad X\theta = y.$$

- Geometric view:



# Why Min-Norm: Because of the Robustness

- Suppose  $y$  changes slightly:  $y \mapsto y + \delta y$ .

# Why Other Solutions Are Less Stable

- Any other interpolant can be written as

$$\theta = \theta^* + v, \quad v \in \mathcal{N}(X).$$

- $v$  does not affect  $X\theta = y$ , but can have large magnitude.
- If  $y$  changes slightly, the new affine set  $\{\theta : X\theta = y + \delta y\}$  shifts.
- So if you have a data-dependent (i.e., dependent upon  $(X, y + \delta y)$ ) selection rule for picking solutions then  $v(\delta y)$  could vary wildly.
- The **minimum-norm** solution means *no nullspace component*  $\implies$  most stable fit.
- For predictions and for perturbations in  $X$ : carrying extra nullspace mass is undesirable; minimum-norm avoids it and provides a clean inductive bias.



# Summary: Robustness of the Minimum-Norm Solution

- All exact fits satisfy  $X\theta = y$ , but they differ in sensitivity to data.
- The minimum-norm solution

$$\theta^* = X^\top (XX^\top)^{-1}y$$

is **most robust**:

- ▶ It changes smoothly with  $y$ .
- ▶ It contains no nullspace component that can amplify noise.
- ▶ It provides the “simplest” exact fit.
- This principle underlies regularization and implicit bias in modern ML: *prefer simple (low-norm), stable solutions.*

# From Stability to Regularization

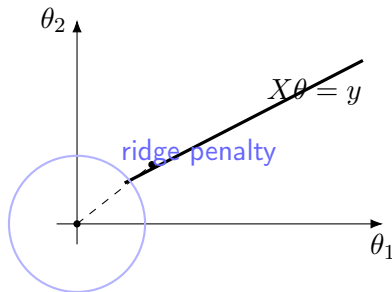
- The minimum-norm solution is a **stable** interpolant:

$$\theta^* = \arg \min_{\theta} \|\theta\|_2^2 \quad \text{s.t.} \quad X\theta = y.$$

- In practice, data are noisy — exact interpolation can overfit.
- General Idea: **trade off fit and simplicity via regularization.**

# Regularization = Controlled Stability

- Regularization keeps the solution **stable** when  $y$  changes slightly.
- It also reduces **variance** and improves **prediction quality**.
- Gradient descent with small initialization often behaves as if  $\lambda$  is small but positive (implicit regularization).



*Geometrically: regularization picks the point on the affine set  $X\theta \approx y$  closest to the smallest-norm ball.*

# Why Regularization Improves Prediction

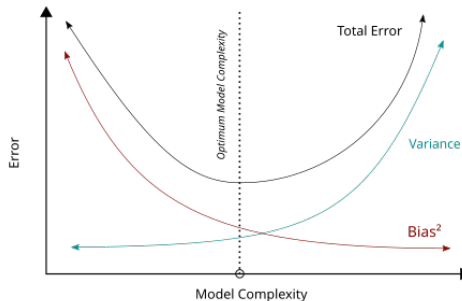
- Model:  $y = X\theta^* + \varepsilon$ ,  $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$
- Ridge solution:

$$\hat{\theta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y.$$

- Bias–variance decomposition:

$$\mathbb{E}\|\hat{y} - X\theta^*\|^2 = \underbrace{\|X(\mathbb{E}[\hat{\theta}_\lambda] - \theta^*)\|^2}_{\text{bias}^2} + \underbrace{\text{tr}(X \text{Var}[\hat{\theta}_\lambda] X^\top)}_{\text{variance}} + \text{noise}.$$

- Increasing  $\lambda$ :
  - ▶  $\uparrow$  bias (shrink coefficients)
  - ▶  $\downarrow$  variance (less noise amplification)
- Optimal  $\lambda$  minimizes total error  $\Rightarrow$  better generalization.



# Gradient Descent Dynamics in Least Squares

- Objective:

$$f(\theta) = \frac{1}{2}\|X\theta - y\|^2, \quad \nabla f(\theta) = X^\top(X\theta - y).$$

# Unrolling the Gradient Descent Recursion

# Unrolling the Gradient Descent Recursion

# Using the Matrix Geometric Series

$$\theta_t = \eta \sum_{k=0}^{t-1} (I - \eta X^\top X)^k X^\top y.$$

- Let  $B = X^\top X$ . The finite geometric series gives

$$\sum_{k=0}^{t-1} (I - \eta B)^k = (\eta B)^{-1} [I - (I - \eta B)^t].$$



# Finite Steps $\implies$ Implicit Regularization

- For a finite number of steps  $t$ ,

$$\theta_t = (X^\top X)^{-1} [I - (I - \eta X^\top X)^t] X^\top y.$$

- For small  $\eta X^\top X$ ,

$$(I - \eta X^\top X)^t \approx (I + \eta t X^\top X)^{-1}.$$

- Hence approximately:

$$\theta_t \approx \left( X^\top X + \frac{1}{\eta t} I \right)^{-1} X^\top y.$$

- This has the same form as **ridge regression**:

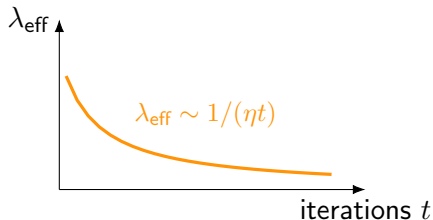
$$\theta_\lambda = (X^\top X + \lambda I)^{-1} X^\top y, \quad \text{with } \lambda_{\text{eff}} \approx \frac{1}{\eta t}.$$

# Interpretation: “Implicit Regularization”

- Even though we did *not* add an explicit penalty, gradient descent acts as if there were a small ridge term.
- **Early stopping** corresponds to a larger effective  $\lambda$ : smoother, smaller-norm solution.
- **Running to convergence** corresponds to  $\lambda_{\text{eff}} \rightarrow 0$ : recovers the minimum-norm interpolant.

Gradient descent with small initialization behaves like ridge regression with  $\lambda_{\text{eff}} \approx \frac{1}{\eta t}$ .

*Stopping gradient descent early acts like adding an  $\ell_2$  penalty: controls model complexity without explicit regularization.*



more training  $\implies$  less regularization

# From Linear Models to Kernels

- For linear regression, “simple” means small  $\|\theta\|$ .
- For nonlinear functions, we need a notion of “smoothness” or “simplicity” in function space.
- In a Reproducing Kernel Hilbert Space (RKHS):

$$\min_{f \in \mathcal{H}} \sum_{i=1}^m (f(x_i) - y_i)^2 + \lambda \|f\|_{\mathcal{H}}^2.$$

- The solution (by the Representer Theorem) is

$$f_{\lambda}(x) = k(x)^{\top} (K + \lambda I)^{-1} y, \quad K_{ij} = k(x_i, x_j).$$

- Same principle:
  - ▶ Fit the data (small residuals),
  - ▶ Keep the function “small” in  $\mathcal{H}$  (smooth, stable).

# Unifying Principle

## Minimum-Norm and Regularization

Everywhere we've looked:

Linear least squares:  $\min_{\theta} \|X\theta - y\|^2 + \lambda\|\theta\|^2,$

Kernel regression:  $\min_{f \in \mathcal{H}} \sum_i (f(x_i) - y_i)^2 + \lambda\|f\|_{\mathcal{H}}^2.$

- Both choose the **most stable, simplest** fit consistent with the data.
- $\lambda$  tunes how much we trust data vs how much we smooth.
- As  $\lambda \downarrow 0$ , we recover the minimum-norm interpolant.

*The core idea: prefer simple (low-norm, smooth) solutions—  
they're the most stable and the most likely to generalize.*